

Informe de Pruebas de Integracion

1. Datos generales

- Asignatura: Aseguramiento de la Calidad
- Taller: Pruebas de Integracion en Arquitecturas Basadas en Mensajes y Eventos
- Estudiante: Ziuvar Ruiz Alvarez
- Fecha de ejecucion: 2026-05-03
- Repositorio: PRUEBAS-INTEGRACION-ASEGURAMIENTO

2. Arquitectura implementada

2.1 RabbitMQ

- Productor: orders-api
- Broker: rabbitmq
- Consumidor: notification-worker
- Cola: pedidos
- Compose: ../docker-compose.rabbitmq.yml

2.2 Kafka

- Productor: orders-api
- Broker: kafka
- Consola: kafka-ui en <http://localhost:8081>
- Topic: orders.events
- Consumidores:
- inventory-consumer
- billing-consumer
- notification-consumer
- Compose: ../docker-compose.kafka.yml

3. Evidencia del entorno

RabbitMQ

- Contenedores y puertos: [03-ps.txt](evidence/latest/rabbitmq/03-ps.txt)
- Salud del productor: [02-health.json](evidence/latest/rabbitmq/02-health.json)
- Logs de arranque del productor: [04-orders-api-start.log](evidence/latest/rabbitmq/04-orders-api-start.log)
- Logs de arranque del consumidor: [05-notification-start.log](evidence/latest/rabbitmq/05-notification-start.log)
- Estado inicial de la cola: [06-queue-state-initial.txt](evidence/latest/rabbitmq/06-queue-state-initial.txt)

Kafka

- Contenedores y puertos: [03-ps.txt](evidence/latest/kafka/03-ps.txt)
- Salud del productor: [02-health.json](evidence/latest/kafka/02-health.json)
- Topic creado con 2 particiones: [04-topic-describe.txt](evidence/latest/kafka/04-topic-describe.txt)
- Logs de arranque del productor: [05-orders-api-start.log](evidence/latest/kafka/05-orders-api-start.log)
- Logs de arranque de consumidores:
- [06-inventory-start.log](evidence/latest/kafka/06-inventory-start.log)
- [07-billing-start.log](evidence/latest/kafka/07-billing-start.log)
- [08-notification-start.log](evidence/latest/kafka/08-notification-start.log)

4. Casos de prueba

CP-01 Creacion de pedido via API

RabbitMQ

- Objetivo: validar que POST /orders recibe la solicitud y responde 202 Accepted.
- Datos de entrada:


```
{ "customerName": "Pedro", "product": "Libro de arquitectura", "quantity": 1 }
```
- Resultado obtenido: la API respondió 202 en 0.008838 s con orderId=ORD-1777838559240.
- Evidencia:
- [07-cp01-response.txt](evidence/latest/rabbitmq/07-cp01-response.txt)
- Estado: Cumple.
- Analisis tecnico: el productor confirma la recepcion sin esperar el procesamiento del worker.

Kafka

- Objetivo: validar que POST /orders recibe la solicitud y responde 202 Accepted.
- Datos de entrada:

```
{"customerName":"Pedro","product":"Libro de arquitectura","quantity":1,"unitPrice":85000}
```

- Resultado obtenido: la API respondió 202 en 0.023589 s con orderId=ORD-1777838599216.
- Evidencia:
- [09-cp01-response.txt](evidence/latest/kafka/09-cp01-response.txt)
- Estado: Cumple.
- Analisis tecnico: el productor publica el evento y responde antes del trabajo de los consumidores.

CP-02 Envio de mensaje a RabbitMQ

- Objetivo: verificar que el pedido genera un mensaje en la cola pedidos.
- Resultado obtenido: el productor publico el pedido ORD-1777838559240 y la cola quedo disponible para consumo.
- Evidencia:
- [08-orders-api-after-cp01.log](evidence/latest/rabbitmq/08-orders-api-after-cp01.log)
- [10-queue-state-after-cp01.txt](evidence/latest/rabbitmq/10-queue-state-after-cp01.txt)
- Estado: Cumple.
- Analisis tecnico: el log del productor muestra el correlationId=32f16a65-36e7-417a-8f7b-82f50aed013c asociado al orderId.

CP-03 Consumo de mensaje (RabbitMQ)

- Objetivo: validar que notification-worker procesa y confirma el mensaje.
- Resultado obtenido: el worker recibio el pedido ORD-1777838559240, simulo el envio de notificacion y realizo ack manual.
- Evidencia:
- [09-notification-after-cp01.log](evidence/latest/rabbitmq/09-notification-after-cp01.log)
- Estado: Cumple.
- Analisis tecnico: el consumidor trabaja desacoplado y deja evidencia del ack al finalizar.

CP-04 Publicacion de evento en Kafka

- Objetivo: verificar que la API publica el evento OrderCreated en orders.events.
- Resultado obtenido: el productor publico orderId=ORD-1777838599216 con correlationId=06f45e2c-7222-48ea-90c1-5f44aba19d77.
- Evidencia:
- [10-orders-api-after-cp01.log](evidence/latest/kafka/10-orders-api-after-cp01.log)

- [04-topic-describe.txt](evidence/latest/kafka/04-topic-describe.txt)
- Estado: Cumple.
- Analisis tecnico: el topic quedo creado con 2 particiones, adecuado para trazabilidad y elasticidad.

CP-05 Consumo de evento (Kafka)

- Objetivo: validar que los tres consumidores reciben el mismo evento.
- Resultado obtenido: inventory-consumer, billing-consumer y notification-consumer procesaron ORD-1777838599216.
- Evidencia:
 - [11-inventory-after-cp01.log](evidence/latest/kafka/11-inventory-after-cp01.log)
 - [12-billing-after-cp01.log](evidence/latest/kafka/12-billing-after-cp01.log)
 - [13-notification-after-cp01.log](evidence/latest/kafka/13-notification-after-cp01.log)
- Estado: Cumple.
- Analisis tecnico: cada consumidor pertenece a un grupo distinto y ejecuta una responsabilidad diferente sobre el mismo evento.

CP-06 Trazabilidad del flujo

RabbitMQ

- Resultado obtenido: el orderId=ORD-1777838559240 y el correlationId=32f16a65-36e7-417a-8f7b-82f50aed013c aparecen tanto en el productor como en el consumidor.
- Evidencia:
 - [08-orders-api-after-cp01.log](evidence/latest/rabbitmq/08-orders-api-after-cp01.log)
 - [09-notification-after-cp01.log](evidence/latest/rabbitmq/09-notification-after-cp01.log)
- Estado: Cumple.

Kafka

- Resultado obtenido: el orderId=ORD-1777838599216 y el correlationId=06f45e2c-7222-48ea-90c1-5f44aba19d77 aparecen en productor y consumidores.
- Evidencia:
 - [10-orders-api-after-cp01.log](evidence/latest/kafka/10-orders-api-after-cp01.log)
 - [11-inventory-after-cp01.log](evidence/latest/kafka/11-inventory-after-cp01.log)
 - [12-billing-after-cp01.log](evidence/latest/kafka/12-billing-after-cp01.log)
 - [13-notification-after-cp01.log](evidence/latest/kafka/13-notification-after-cp01.log)
- Estado: Cumple.

CP-07 Responsive

- Objetivo: validar que la API responde sin esperar el trabajo del consumidor.
- Resultado obtenido:
- RabbitMQ: respuesta en 0.008838 s, mientras el worker procesa despues en los logs.
- Kafka: respuesta en 0.023589 s, mientras los consumidores procesan despues en los logs.
- Evidencia:
- [07-cp01-response.txt](evidence/latest/rabbitmq/07-cp01-response.txt)
- [09-notification-after-cp01.log](evidence/latest/rabbitmq/09-notification-after-cp01.log)
- [09-cp01-response.txt](evidence/latest/kafka/09-cp01-response.txt)
- [11-inventory-after-cp01.log](evidence/latest/kafka/11-inventory-after-cp01.log)
- Estado: Cumple.
- Analisis tecnico: la integracion es asincronica en ambos stacks; la API solo depende del broker.

CP-08 Message-driven

- Objetivo: verificar que no existe comunicacion HTTP directa entre productor y consumidores.
- Resultado obtenido: los consumidores no exponen endpoints HTTP y toda la comunicacion fluye por broker.
- Evidencia:
- ../rabbitmq/notification-worker/worker.js
- ../kafka/consumer/consumer.js
- ../docker-compose.rabbitmq.yml
- ../docker-compose.kafka.yml
- Estado: Cumple.
- Analisis tecnico: solo los productores publican al broker; los consumidores unicamente se suscriben o consumen colas/topics.

CP-09 Resilient

RabbitMQ

- Objetivo: validar el comportamiento al apagar el consumidor.
- Resultado obtenido: con el worker detenido, el pedido ORD-1777838563191 siguio entrando y la cola quedo con messages_ready=1.
- Evidencia:
- [12-cp09-response.txt](evidence/latest/rabbitmq/12-cp09-response.txt)

- [14-queue-state-worker-stopped.txt](evidence/latest/rabbitmq/14-queue-state-worker-stopped.txt)
- Estado: Cumple.
- Analisis tecnico: el productor siguio operando aunque no existiera consumidor activo.

Kafka

- Objetivo: validar el comportamiento al apagar notification-consumer.
- Resultado obtenido: el pedido ORD-1777838602877 fue publicado con el consumidor detenido y el grupo mostro LAG=1.
- Evidencia:
- [16-cp09-response.txt](evidence/latest/kafka/16-cp09-response.txt)
- [21-consumer-groups-while-stopped.txt](evidence/latest/kafka/21-consumer-groups-while-stopped.txt)
- [17-notification-while-stopped.log](evidence/latest/kafka/17-notification-while-stopped.log)
- Estado: Cumple.
- Analisis tecnico: el evento quedo retenido en Kafka hasta que el consumidor del mismo grupo regreso.

CP-10 Elastic

RabbitMQ

- Objetivo: verificar distribucion de carga con multiples consumidores.
- Resultado obtenido: al escalar notification-worker=2, ambos contenedores procesaron pedidos diferentes.
- Evidencia:
- [19-ps-scaled.txt](evidence/latest/rabbitmq/19-ps-scaled.txt)
- [21-notification-after-scale.log](evidence/latest/rabbitmq/21-notification-after-scale.log)
- Estado: Cumple.
- Analisis tecnico: la cola distribuyo trabajo entre notification-worker-1 y notification-worker-2.

Kafka

- Objetivo: verificar distribucion de particiones con dos instancias del mismo grupo.
- Resultado obtenido: el grupo notification-service-group quedo con una instancia en la particion 0 y otra en la 1.
- Evidencia:
- [25-ps-scaled.txt](evidence/latest/kafka/25-ps-scaled.txt)
- [28-consumer-group-after-scale.txt](evidence/latest/kafka/28-consumer-group-after-scale.txt)
- [27-notification-after-scale.log](evidence/latest/kafka/27-notification-after-scale.log)

- Estado: Cumple.
- Analisis tecnico: la elasticidad depende de particiones, no de una cola compartida.

CP-11 Desacoplamiento temporal

RabbitMQ

- Resultado obtenido: con el consumidor apagado, el productor publico ORD-1777838563191 y el mensaje quedo esperando en la cola.
- Evidencia:
 - [12-cp09-response.txt](evidence/latest/rabbitmq/12-cp09-response.txt)
 - [14-queue-state-worker-stopped.txt](evidence/latest/rabbitmq/14-queue-state-worker-stopped.txt)
- Estado: Cumple.

Kafka

- Resultado obtenido: con notification-consumer detenido, el productor publico ORD-1777838602877 y el grupo acumulo rezago hasta el reinicio.
- Evidencia:
 - [16-cp09-response.txt](evidence/latest/kafka/16-cp09-response.txt)
 - [21-consumer-groups-while-stopped.txt](evidence/latest/kafka/21-consumer-groups-while-stopped.txt)
- Estado: Cumple.

CP-12 Recuperacion

RabbitMQ

- Resultado obtenido: al reiniciar el worker, el pedido ORD-1777838563191 se consumio y la cola volvio a 0.
- Evidencia:
 - [15-start-worker.txt](evidence/latest/rabbitmq/15-start-worker.txt)
 - [16-notification-after-recovery.log](evidence/latest/rabbitmq/16-notification-after-recovery.log)
 - [17-queue-state-after-recovery.txt](evidence/latest/rabbitmq/17-queue-state-after-recovery.txt)
- Estado: Cumple.

Kafka

- Resultado obtenido: al reiniciar notification-consumer, el evento ORD-1777838602877 se proceso despues del retorno del servicio.
- Evidencia:

- [22-start-notification.txt](evidence/latest/kafka/22-start-notification.txt)
- [23-notification-after-recovery.log](evidence/latest/kafka/23-notification-after-recovery.log)
- Estado: Cumple.

5. Conclusiones

5.1 Que arquitectura fue mas robusta desde QA

RabbitMQ fue mas directa para validar desde QA cuando el foco fue observar cola, ack y retencion inmediata. Kafka fue mas robusta para fan-out, trazabilidad por offsets y escalado por particiones, pero exigio una lectura mas tecnica de consumer groups y del broker.

5.2 Que dificultades se presentaron durante las pruebas

- La imagen original bitnami/kafka:3.7 ya no estaba disponible, por lo que se actualizo a bitnamilegacy/kafka:4.0.0-debian-12-r10.
- El puerto 8080 estaba ocupado en el equipo de ejecucion, por lo que Kafka UI se expuso en 8081.
- Kafka requirio mayor tiempo de warm-up del broker y del group coordinator antes de quedar listo para pruebas funcionales.

5.3 Que diferencias reales se evidenciaron entre RabbitMQ y Kafka

- RabbitMQ trabaja mejor para mensajeria orientada a cola y reparto directo de trabajo.
- Kafka conserva el evento como log y permite relectura, fan-out por grupos y evidencia fuerte de offsets/lag.
- La elasticidad en RabbitMQ se observo por alternancia de workers; en Kafka se observo por asignacion de particiones a miembros distintos del mismo grupo.

6. Anexos

- Resumen automatizado: [summary.json](evidence/latest/summary.json)
- Evidencia RabbitMQ: evidence/latest/rabbitmq/
- Evidencia Kafka: evidence/latest/kafka/
- Script de recoleccion: ./scripts/collect-evidence.ps1